

Corporate Equipment Allocation and Tracking System

1. Introduction

The Corporate Equipment Allocation and Tracking System is designed to streamline the distribution and tracking of office equipment, such as laptops, projectors, and tablets. Reception staff can efficiently allocate equipment, track its availability, and ensure timely returns. The system also automates notifications to employees, maintenance, and inventory teams, improving operational efficiency and accountability.

2. Problem Statement

Managing office equipment manually can lead to delays, misplaced items, and missed maintenance requirements. Reception staff need a system to:

- Record equipment allocations and returns.
- Track employee usage and expected return dates.
- Automate reminders for overdue returns.
- Alert maintenance for damaged equipment.
- Keep inventory records up-to-date.

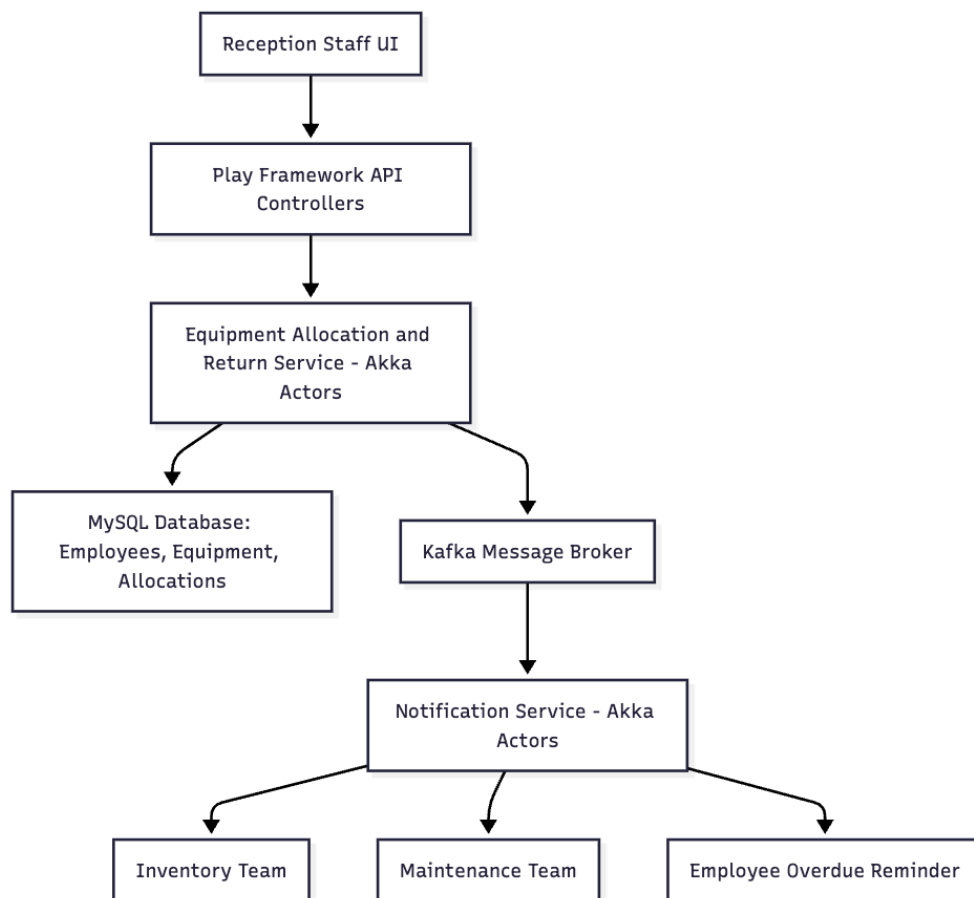
This system aims to solve these challenges by providing a reliable, scalable, and automated solution.

3. Architecture Overview

The system follows a **modular microservice-based architecture**. The key components include:

1. **Frontend / Application Layer** – Interfaces for reception staff to request allocation/return and view equipment status.
2. **API Layer (Play Framework)** – Handles REST requests for allocation, return, and status updates.

3. **Service Layer** – Contains business logic, validation, and workflows.
4. **Database Layer (MySQL)** – Stores employees, equipment, and allocation records.
5. **Notification Service (Akka Actors)** – Handles asynchronous notifications like overdue reminders and maintenance alerts.
6. **Message Broker (Kafka)** – Queues notifications to ensure reliable delivery.
7. **Deployment** – Docker containers for consistent and scalable deployment.



4. Component Design

Frontend / App:

- User interface for reception staff.
- Submit allocation and return requests.
- View status of equipment.

Play API Controllers:

- Receive and validate REST API requests.
- Pass data to the Service Layer.

Service Layer:

- Apply business logic for allocation and return.
- Trigger notifications based on events (overdue, damage, etc.).

Database (MySQL):

- Tables: Employees, Equipment, Allocations.
- Stores allocation history and status.

Akka Notification Service:

- Actors manage asynchronous notifications.
- Schedulers handle reminders for overdue equipment.
- Kafka consumers listen for events from the database/service layer.

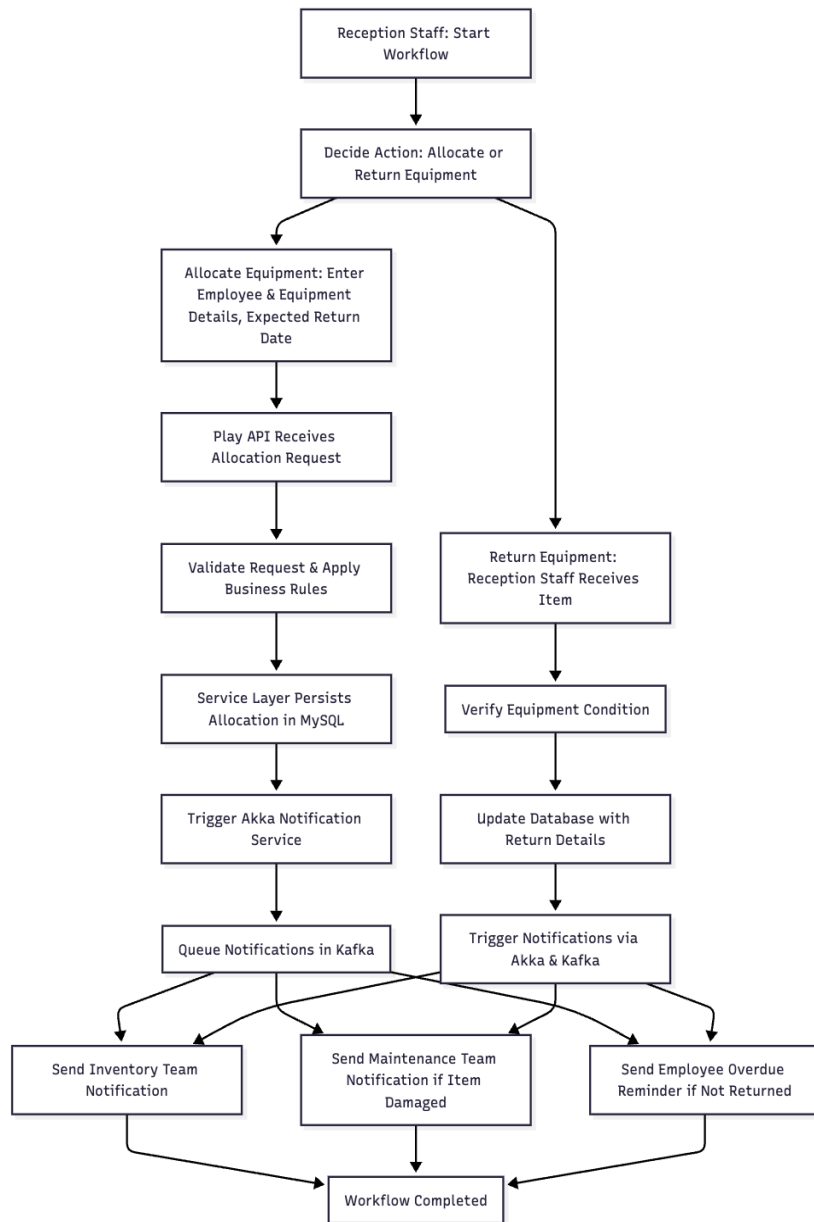
Kafka (Messaging Layer):

- Queue notifications for delivery to email services.
- Ensures no notifications are lost even if services are busy.

Docker Deployment:

- Each component runs in its own container for consistency.
- Supports scalability and easy environment replication.

5. System Workflow



6. Microservices Design

The system implements a microservice architecture for modularity and scalability.

Microservices and Purpose

- Equipment Allocation Service:
 - Handles allocation, return, and equipment status tracking.
 - Updates the database with allocation and return information.
 - Applies business rules to ensure equipment availability.
- Notification Service:
 - Sends reminders to employees for overdue equipment.
 - Alerts maintenance for damaged items.
 - Notifies inventory about allocations and returns.

Inter-Service Communication

- Synchronous: Frontend → Equipment Allocation Service via REST API.
- Asynchronous: Equipment Allocation Service → Notification Service via Kafka.

Design Rationale:

- Separating notification logic from allocation ensures high responsiveness.
- Independent scaling is possible for services under heavy load.
- Improves maintainability and modularity.

7. REST API Endpoints

The Corporate Equipment Allocation and Tracking System exposes the following REST APIs to manage equipment allocation, return, and status tracking. These endpoints are primarily consumed by the frontend application used by reception staff.

1. Equipment Allocation Service

- **POST /allocate**
 - **Purpose:** Allocate equipment to an employee.
 - **Input Parameters:**
 - Employee details (Name, Department, Purpose)
 - Equipment ID(s)
 - Allocation date
 - Expected return date
 - **Behavior:**
 - Validates employee and equipment details.
 - Persists allocation in the database.
 - Triggers notifications for inventory tracking and overdue reminders.
- **POST /return**
 - **Purpose:** Register the return of allocated equipment.
 - **Input Parameters:**
 - Equipment ID(s)
 - Condition of the equipment (e.g., good, damaged)
 - Return date
 - **Behavior:**
 - Updates the allocation record in the database.
 - Checks equipment condition and triggers maintenance notifications if required.
 - Sends notifications to the inventory team and employee (if overdue reminders are needed).

- **GET /status/{employeeId}**
 - **Purpose:** Retrieve the allocation status and history for a specific employee.
 - **Input Parameters:** Employee ID
 - **Behavior:**
 - Fetches allocation records from the database.
 - Provides details such as current equipment held, allocation date, expected return date, and past allocation history.

2. Notification Service

- **Purpose:** Handle all automated notifications related to equipment allocation and return.
- **Endpoints:** No public REST endpoints.
- **Behavior:**
 - Consumes events from Kafka triggered by allocation or return actions.
 - Sends notifications via email/SMS to employees, inventory, and maintenance teams.
 - Ensures asynchronous, reliable delivery of reminders and alerts.

7. Security

- **Authentication:** JWT-based tokens for secure, stateless user authentication.
- **Authorization:** Role-based access control.
 - Reception Staff: Allocate/return equipment, view status.
 - Inventory Team: Receive notifications, view inventory.
 - Maintenance Team: Receive notifications, track damaged equipment.
- **Transport Security:** HTTPS for all communications to encrypt sensitive data.

8. Testing Strategy

- Unit Testing:
 - Test allocation and return logic independently.
 - Validate notification triggers.
- Integration Testing:
 - Test end-to-end workflow from allocation to notification.
 - Verify database updates and message delivery through Kafka.

9. Key Design Decisions

- **Microservice Architecture:** Enables independent scaling of API, Notification Service, and database.
- **Kafka Messaging:** Ensures reliable, asynchronous notifications.
- **Akka Actors:** Efficiently handle scheduled and real-time notifications.
- **Dockerization:** Guarantees consistent deployment across environments.
- **Database Normalization:** Prevents duplication of equipment and employee data.
- **REST API:** Standardized communication between frontend and backend.

10. Conclusion

The proposed architecture provides a **scalable, reliable, and automated solution** for corporate equipment management. It ensures:

- Timely allocation and return of equipment.
- Automated notifications to employees and internal teams.
- Accurate tracking of inventory and maintenance needs.
- Maintainable and extensible system design for future expansion.